
Seguro Viagem Gestão de Apólices

Documentação técnica do código

Laravel 12 · React 18 · MySQL 8 · Microsoft Azure

Carlos Guilherme Fontes Pereira

Teste técnico – CORIS

23 de setembro de 2026

Sumário

1	Visão geral	3
1.1	Requisitos do teste e onde são atendidos	3
1.2	Tecnologias	3
1.3	Estrutura de pastas	3
2	Arquitetura	4
2.1	Camadas do backend	4
2.2	Fluxo completo: emitir uma apólice	4
2.3	Solução na Azure	4
3	Modelo de dados	5
4	Regras de negócio	7
4.1	Cálculo do prêmio	7
4.2	Outras regras	7
5	Backend: código comentado	8
5.1	Rotas	8
5.2	Enums do domínio	8
5.3	Validação: Form Requests e regra de CPF	10
5.4	Controllers	13
5.5	Service: regras de negócio	15
5.6	Calculadora do prêmio	17
5.7	Models	19
5.8	Resource: formato da resposta	21
5.9	Tratamento de erros	22
6	Referência da API	22
7	Frontend	24
7.1	Telas e rotas	24
7.2	Comunicação com a API	24
7.3	Cotação em tempo real	26
7.4	Opções e formatação	26
8	SOLID e Clean Code	27

9	Testes automatizados	28
9.1	Testes de unidade	28
9.2	Testes da API	29
10	Execução	29
10.1	Docker (modo principal)	29
10.2	Sem Docker	30
10.3	Publicação na Azure	30
11	Próximos passos	30

1 Visão geral

A aplicação é um sistema web para **gestão de apólices de seguro viagem**. Um operador cadastra o segurado, escolhe o destino, o plano e o período da viagem, e o sistema calcula o *prêmio* (o valor pago pelo seguro) em tempo real. As apólices podem ser consultadas com busca e paginação, editadas (com recálculo do prêmio), canceladas, reativadas e excluídas.

1.1 Requisitos do teste e onde são atendidos

Requisito	Onde está no código
CRUD completo	ApoliceController, ApoliceService e as telas em frontend/src/pages
SOLID e Clean Code	Separação em Form Request, Controller, Service e Calculadora (Seção 8)
Frontend em React	frontend/ (React 18 + Vite + React Router)
Banco relacional	MySQL 8, tabelas segurados 1:N apolices (Seção 3)
Desenho na Azure	docs/arquitetura-azure.svg (Seção 2.3)
Testes	25 testes PHPUnit em backend/tests (Seção 9)

1.2 Tecnologias

- **Backend:** PHP 8.3 e Laravel 12 (Eloquent, Form Requests, API Resources, SoftDeletes).
- **Frontend:** React 18, Vite e React Router, sem biblioteca de componentes.
- **Banco:** MySQL 8 no Docker e na Azure; SQLite para rodar localmente sem instalar banco.
- **Testes:** PHPUnit (testes de unidade e de API).
- **Infraestrutura:** Docker Compose (MySQL, API em Apache + PHP e frontend em Nginx) e Microsoft Azure.

1.3 Estrutura de pastas

Código 1: Organização do repositório (arquivos principais)

```

backend/
  app/
    Enums/                Plano, Destino, StatusApolice
    Http/Controllers/Api/ ApoliceController, OpcoesController
    Http/Requests/        CotacaoRequest, SalvarApoliceRequest
    Http/Resources/        ApoliceResource (formato do JSON)
    Models/                Segurado, Apolice
    Providers/            AppServiceProvider (injeção de dependência)
    Rules/                 Cpf (validação dos dígitos verificadores)
    Services/              ApoliceService (regras de negócio)
    Services/Premio/        CalculadoraPremio (interface) e CalculadoraPremioViagem
  bootstrap/app.php        tratamento de erros da API
  database/migrations/      criação das tabelas
  database/seeder/          dados de exemplo
  routes/api.php            rotas da API
  tests/                    Feature (API) e Unit (calculadora e CPF)
frontend/
  src/api/                  cliente HTTP e chamadas da API
  src/components/          formulário, paginação, diálogo, toast...
  src/hooks/               useApolice, useOpcoes

```

src/pages/	lista, detalhe, cadastro e edição
src/utils/format.js	moeda, datas e máscara de CPF
docs/	diagrama da Azure e esta documentação
docker-compose.yml	

2 Arquitetura

2.1 Camadas do backend

O backend segue uma arquitetura em camadas. Cada camada tem uma única responsabilidade e só conhece a camada seguinte:

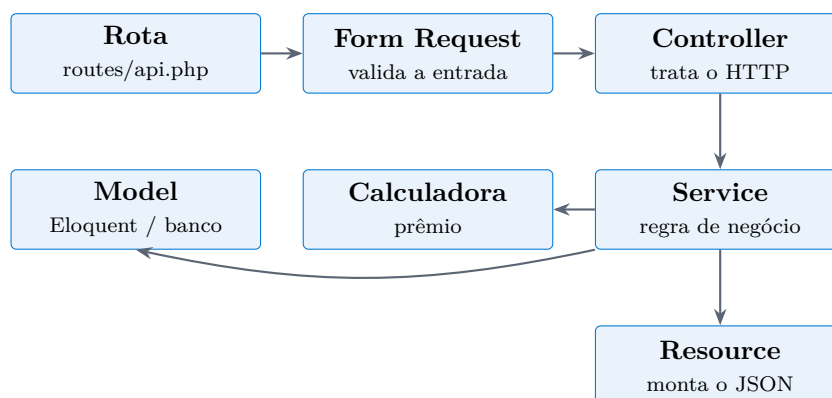


Figura 1: Caminho de uma requisição dentro do backend.

2.2 Fluxo completo: emitir uma apólice

1. Na tela `ApoliceCreate.jsx`, o operador envia o formulário e o front chama `apolicesApi.criar(dados)`.
2. `client.js` faz um `POST /api/apolices` com o corpo em JSON.
3. O Laravel encontra a rota em `routes/api.php` e resolve o `ApoliceController`.
4. Antes de o controller rodar, o `SalvarApoliceRequest` valida os campos. Se algo estiver errado, a resposta é 422 com a mensagem de cada campo, e o controller nem é chamado.
5. `ApoliceController::store` chama `ApoliceService::criar` com os dados já validados.
6. O service, dentro de uma transação, salva ou reaproveita o segurado pelo CPF, calcula o prêmio, gera o número da apólice e grava a apólice.
7. O `ApoliceResource` transforma o model em JSON e a API responde 201 `Created`.
8. O front mostra a notificação e abre a tela de detalhe da nova apólice.

2.3 Solução na Azure

- **Azure Static Web Apps:** hospeda o React já compilado (arquivos estáticos) em CDN, com HTTPS.
- **Azure App Service (Linux):** executa a API Laravel em PHP 8.3 e permite escalar horizontalmente.

Seguro Viagem · Gestão de Apólices — arquitetura na Azure

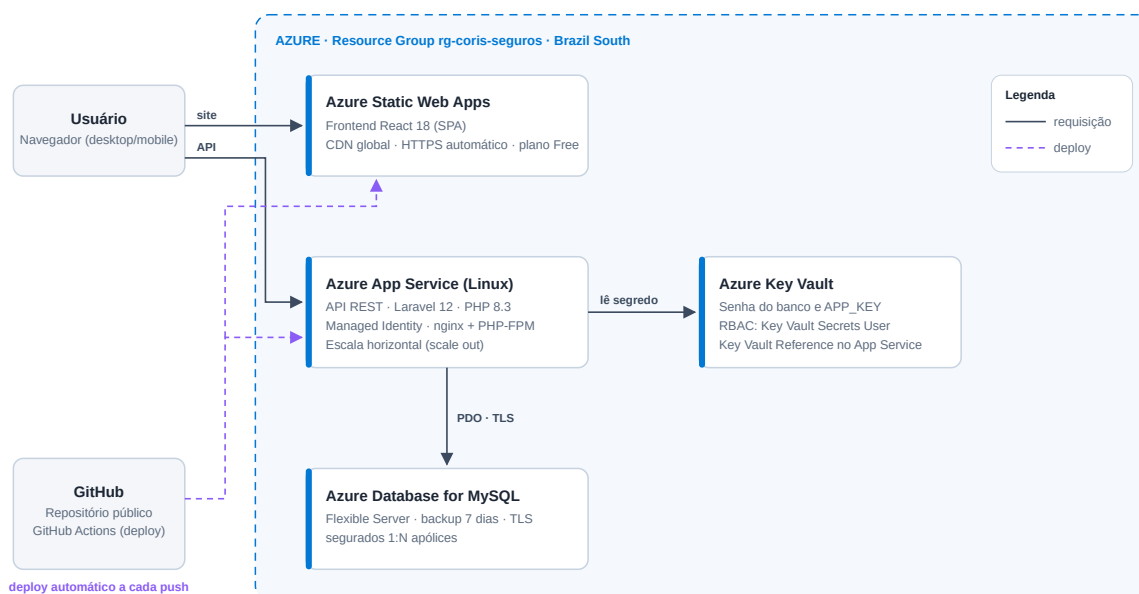


Figura 2: Desenho da solução na Azure (docs/arquitetura-azure.svg).

- **Azure Database for MySQL – Flexible Server:** banco gerenciado, com backup e conexão TLS.
- **Azure Key Vault:** guarda a senha do banco e a APP_KEY. O App Service lê os segredos com *Managed Identity*, então nenhuma senha fica no código.
- **GitHub Actions:** publica a aplicação a cada *push*.

3 Modelo de dados

Há duas tabelas. Um **segurado** pode ter várias **apólices** (relação 1:N). O segurado é identificado pelo CPF, que é único, e é reaproveitado quando a mesma pessoa contrata outro seguro.

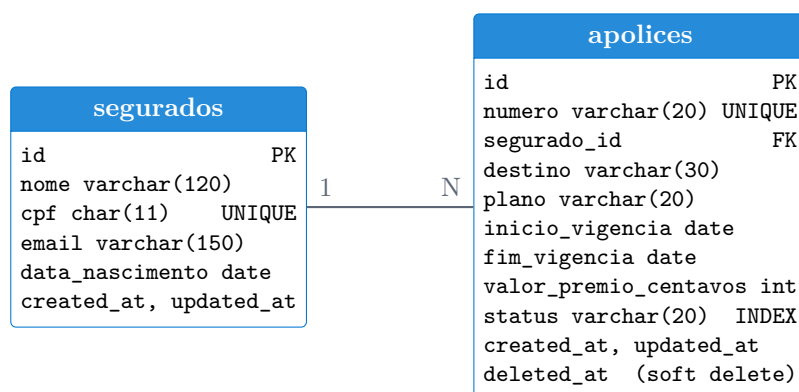


Figura 3: Diagrama entidade-relacionamento.

Decisões de modelagem

- **Dinheiro em centavos (inteiro):** float não representa valores decimais com exatidão ($0,1 + 0,2 \neq 0,3$). Em seguros, um centavo de diferença é um erro de negócio.
- **CPF só com dígitos:** guardado sem máscara (`char(11)`) e formatado apenas na resposta.
- **Exclusão lógica:** a coluna `deleted_at` marca a apólice como excluída sem apagar o registro. Contratos de seguro precisam ficar guardados para auditoria.
- **Índice em status:** o filtro por status e o resumo da tela inicial consultam essa coluna.

Código 2: database/migrations/..._create_segurados_table.php

```
1 <?php
2
3 use Illuminate\Database\Migrations\Migration;
4 use Illuminate\Database\Schema\Blueprint;
5 use Illuminate\Support\Facades\Schema;
6
7 return new class extends Migration
8 {
9     public function up(): void
10     {
11         Schema::create('segurados', function (Blueprint $table) {
12             $table->id();
13             $table->string('nome', 120);
14             $table->char('cpf', 11)->unique();
15             $table->string('email', 150);
16             $table->date('data_nascimento');
17             $table->timestamps();
18         });
19     }
20
21     public function down(): void
22     {
23         Schema::dropIfExists('segurados');
24     }
25 };
```

Código 3: database/migrations/..._create_apolices_table.php

```
1 <?php
2
3 use Illuminate\Database\Migrations\Migration;
4 use Illuminate\Database\Schema\Blueprint;
5 use Illuminate\Support\Facades\Schema;
6
7 return new class extends Migration
8 {
9     public function up(): void
10     {
11         Schema::create('apolices', function (Blueprint $table) {
12             $table->id();
13             $table->string('numero', 20)->unique();
14             $table->foreignId('segurado_id')->constrained('segurados');
15             $table->string('destino', 30);
16             $table->string('plano', 20);
17             $table->date('inicio_vigencia');
18             $table->date('fim_vigencia');
19             $table->unsignedInteger('valor_premio_centavos');
20             $table->string('status', 20)->default('ativa')->index();
21             $table->timestamps();
22         });
23     }
24 }
```

```

22     $table->softDeletes();
23     });
24 }
25
26 public function down(): void
27 {
28     Schema::dropIfExists('apolices');
29 }
30 };

```

4 Regras de negócio

4.1 Cálculo do prêmio

O prêmio é calculado pela fórmula:

$$\text{prêmio} = \text{diária do plano} \times \text{dias de viagem} \times \% \text{ do destino} \times \% \text{ da idade}$$

Os dias contam o início e o fim da viagem (de 01/10 a 10/10 são 10 dias). A idade é calculada na data de **início** da viagem, e não na data do cadastro.

Plano	Diária	Cobertura médica	Destino	%
Essencial	R\$ 12,90	R\$ 30.000	Nacional	50%
Plus	R\$ 24,90	R\$ 60.000	América do Sul	100%
Premium	R\$ 39,90	R\$ 150.000	Europa	130%
			América do Norte	140%
			Ásia, África, Oceania	150%

Idade	%
até 59 anos	100%
60 a 74 anos	160%
75 anos ou mais	250%

Exemplo

Plano Plus, Europa, 10 dias, segurado com 31 anos: $2.490 \times 10 = 24.900$ centavos; $\times 130\% = 32.370$; $\times 100\% = 32.370$ centavos, ou seja, **R\$ 323,70**. O mesmo cenário com um segurado de 66 anos fica $32.370 \times 160\% = 51.792$ centavos (R\$ 517,92).

Cada percentual é aplicado com `intdiv(centavos * percentual + 50, 100)`: a soma de 50 antes da divisão inteira arredonda para o centavo mais próximo, sem passar por `float`.

4.2 Outras regras

- **CPF** validado pelos dois dígitos verificadores; sequências repetidas (111.111.111-11) são recusadas.
- **Vigência:** o fim deve ser igual ou posterior ao início, e o período tem no máximo 365 dias.

- **Início da vigência** não pode ser anterior a hoje. Na edição, a regra só vale se a data de início foi alterada; assim, uma apólice que já está em vigor continua editável.
- **Segurado reaproveitado:** ao cadastrar um CPF que já existe, os dados do segurado são atualizados e a nova apólice é ligada a ele.
- **Número da apólice** no formato CRS-2026-XXXXXXX (ano + 8 caracteres hexadecimais aleatórios).
- **Apólice cancelada** só pode ser alterada para voltar a ficar ativa.
- **Exclusão lógica:** a apólice some das consultas, mas continua no banco.

5 Backend: código comentado

5.1 Rotas

As rotas específicas (`resumo` e `cotacao`) são declaradas *antes* do `apiResource`. O `whereNumber('apolice')` garante que `/apolices/resumo` não seja confundido com `/apolices/{id}`.

Código 4: routes/api.php

```
1 <?php
2
3 use App\Http\Controllers\Api\ApoliceController;
4 use App\Http\Controllers\Api\OpcoesController;
5 use Illuminate\Support\Facades\Route;
6
7 Route::get('/health', fn () => ['status' => 'ok']);
8 Route::get('/opcoes', OpcoesController::class);
9
10 Route::get('/apolices/resumo', [ApoliceController::class, 'resumo']);
11 Route::post('/apolices/cotacao', [ApoliceController::class, 'cotacao']);
12 Route::apiResource('apolices', ApoliceController::class)
13     ->parameters(['apolices' => 'apolice'])
14     ->whereNumber('apolice');
```

5.2 Enums do domínio

Planos, destinos e status são *enums* do PHP 8.1+. Cada enum guarda o próprio texto de exibição e os seus valores de negócio (diária, cobertura, percentual de risco). Para criar um plano novo basta adicionar um `case`; o frontend recebe a lista pela rota `/api/opcoes` e o exibe sem mudança no React.

Código 5: app/Enums/Plano.php

```
1 <?php
2
3 namespace App\Enums;
4
5 enum Plano: string
6 {
7     case Essencial = 'essencial';
8     case Plus = 'plus';
9     case Premium = 'premium';
10 }
```

```
11 public function label(): string
12 {
13     return match ($this) {
14         self::Essencial => 'Essencial',
15         self::Plus => 'Plus',
16         self::Premium => 'Premium',
17     };
18 }
19
20 public function diariaCentavos(): int
21 {
22     return match ($this) {
23         self::Essencial => 1290,
24         self::Plus => 2490,
25         self::Premium => 3990,
26     };
27 }
28
29 public function coberturaMedicaCentavos(): int
30 {
31     return match ($this) {
32         self::Essencial => 3_000_000,
33         self::Plus => 6_000_000,
34         self::Premium => 15_000_000,
35     };
36 }
37 }
```

Código 6: app/Enums/Destino.php

```
1 <?php
2
3 namespace App\Enums;
4
5 enum Destino: string
6 {
7     case Nacional = 'nacional';
8     case AmericaDoSul = 'america_do_sul';
9     case AmericaDoNorte = 'america_do_norte';
10    case Europa = 'europa';
11    case Asia = 'asia';
12    case Africa = 'africa';
13    case Oceania = 'oceania';
14
15    public function label(): string
16    {
17        return match ($this) {
18            self::Nacional => 'Nacional',
19            self::AmericaDoSul => 'América do Sul',
20            self::AmericaDoNorte => 'América do Norte',
21            self::Europa => 'Europa',
22            self::Asia => 'Ásia',
23            self::Africa => 'África',
24            self::Oceania => 'Oceania',
25        };
26    }
27
28    public function percentualRisco(): int
29    {
30        return match ($this) {
31            self::Nacional => 50,
32            self::AmericaDoSul => 100,
33            self::Europa => 130,
```

```

34         self::AmericaDoNorte => 140,
35         self::Asia, self::Africa, self::Oceania => 150,
36     };
37 }
38 }

```

Código 7: app/Enums/StatusApolice.php

```

1  <?php
2
3  namespace App\Enums;
4
5  enum StatusApolice: string
6  {
7      case Ativa = 'ativa';
8      case Cancelada = 'cancelada';
9
10     public function label(): string
11     {
12         return match ($this) {
13             self::Ativa => 'Ativa',
14             self::Cancelada => 'Cancelada',
15         };
16     }
17 }

```

5.3 Validação: Form Requests e regra de CPF

A validação fica fora do controller. O CotacaoRequest tem as regras usadas tanto na cotação quanto no cadastro. O SalvarApoliceRequest herda essas regras e acrescenta o **status** e a verificação do início da vigência. O método `after()` executa as validações que dependem de mais de um campo.

Código 8: app/Http/Requests/CotacaoRequest.php

```

1  <?php
2
3  namespace App\Http\Requests;
4
5  use App\Enums\Destino;
6  use App\Enums\Plano;
7  use App\Rules\Cpf;
8  use Illuminate\Foundation\Http\FormRequest;
9  use Illuminate\Support\Carbon;
10 use Illuminate\Validation\Rule;
11 use Illuminate\Validation\Validator;
12
13 class CotacaoRequest extends FormRequest
14 {
15     public const VIGENCIA_MAXIMA_DIAS = 365;
16
17     public function rules(): array
18     {
19         return [
20             'seguradoNome' => ['required', 'string', 'min:3', 'max:120'],
21             'seguradoCpf' => ['required', new Cpf()],
22             'seguradoEmail' => ['required', 'email', 'max:150'],
23             'seguradoNascimento' => ['required', 'date_format:Y-m-d',

```

```

24         'destino' => ['required', Rule::in(array_column(Destino::cases(), 'value'))],
25         'plano' => ['required', Rule::in(array_column(Plano::cases(), 'value'))],
26         'inicioVigencia' => ['required', 'date_format:Y-m-d'],
27         'fimVigencia' => ['required', 'date_format:Y-m-d',
↵ 'after_or_equal:inicioVigencia'],
28     ];
29 }
30
31 public function after(): array
32 {
33     return [
34         function (Validator $validator) {
35             if ($validator->errors()->hasAny(['inicioVigencia', 'fimVigencia'])) {
36                 return;
37             }
38
39             $dias =
↵ Carbon::parse($this->input('inicioVigencia'))->diffInDays($this->input('fimVigencia'))
↵ + 1;
40
41             if ($dias > self::VIGENCIA_MAXIMA_DIAS) {
42                 $validator->errors()->add('fimVigencia', 'A vigência máxima é de
↵ '.self::VIGENCIA_MAXIMA_DIAS.' dias.');
```

Código 9: app/Http/Requests/SalvarApoliceRequest.php

```

1 <?php
2
3 namespace App\Http\Requests;
4
5 use App\Enums>StatusApolice;
6 use App\Models\Apolice;
7 use Illuminate\Support\Carbon;
8 use Illuminate\Validation\Rule;
9 use Illuminate\Validation\Validator;
10
11 class SalvarApoliceRequest extends CotacaoRequest
12 {
```

```

13 public function rules(): array
14 {
15     return [
16         ...parent::rules(),
17         'status' => ['sometimes', Rule::in(array_column(StatusApolice::cases(),
↵ 'value'))],
18     ];
19 }
20
21 public function after(): array
22 {
23     return [
24         ...parent::after(),
25         function (Validator $validator) {
26             if ($validator->errors()->has('inicioVigencia') || !
↵ $this->inicioFoiInformadoOuAlterado()) {
27                 return;
28             }
29
30             if (Carbon::parse($this->input('inicioVigencia'))->lt(today())) {
31                 $validator->errors()->add('inicioVigencia', 'O início da vigência não
↵ pode ser anterior a hoje.');
```

A regra Cpf calcula os dois dígitos verificadores. Para o primeiro dígito, os 9 primeiros números são multiplicados pelos pesos 10 a 2; para o segundo, os 10 primeiros pelos pesos 11 a 2. O dígito esperado é $(10 \times \text{soma}) \bmod 11$, e o resultado 10 vira 0.

Código 10: app/Rules/Cpf.php

```

1 <?php
2
3 namespace App\Rules;
4
5 use Closure;
6 use Illuminate\Contracts\Validation\ValidationRule;
7
8 class Cpf implements ValidationRule
9 {
10     public function validate(string $attribute, mixed $value, Closure $fail): void
11     {
12         if (! is_string($value) || ! self::valido($value)) {
13             $fail('CPF inválido.');
```

```
21     if (strlen($cpf) !== 11 || preg_match('/^(\d)\1{10}$/', $cpf)) {
22         return false;
23     }
24
25     for ($posicao = 9; $posicao < 11; $posicao++) {
26         $soma = 0;
27         for ($i = 0; $i < $posicao; $i++) {
28             $soma += (int) $cpf[$i] * (($posicao + 1) - $i);
29         }
30
31         if ((int) $cpf[$posicao] !== ((10 * $soma) % 11) % 10) {
32             return false;
33         }
34     }
35
36     return true;
37 }
38 }
```

5.4 Controllers

O controller só recebe a requisição, chama o service e devolve a resposta com o status HTTP correto. Não há regra de negócio nele. Graças ao *route model binding*, o Laravel busca a apólice pelo id da URL e devolve 404 quando ela não existe ou foi excluída.

Código 11: app/Http/Controllers/Api/ApoliceController.php

```
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Http\Controllers\Controller;
6  use App\Http\Requests\CotacaoRequest;
7  use App\Http\Requests\SalvarApoliceRequest;
8  use App\Http\Resources\ApoliceResource;
9  use App\Models\Apolice;
10 use App\Services\ApoliceService;
11 use Illuminate\Http\JsonResponse;
12 use Illuminate\Http\Request;
13 use Illuminate\Http\Resources\Json\AnonymousResourceCollection;
14 use Illuminate\Http\Response;
15
16 class ApoliceController extends Controller
17 {
18     public function __construct(private readonly ApoliceService $service)
19     {
20     }
21
22     public function index(Request $request): AnonymousResourceCollection
23     {
24         return ApoliceResource::collection(
25             $this->service->listar($request->query('busca'), $request->query('status'))
26         );
27     }
28
29     public function resumo(): JsonResponse
30     {
31         return response()->json($this->service->resumo());
32     }
33 }
```

```

34 public function show(Apolice $apolicy): ApoliceResource
35 {
36     return new ApoliceResource($apolicy->load('segurado'));
37 }
38
39 public function store(SalvarApoliceRequest $request): JsonResponse
40 {
41     $apolicy = $this->service->criar($request->validated());
42
43     return (new ApoliceResource($apolicy))->response()->setStatusCode(201);
44 }
45
46 public function update(SalvarApoliceRequest $request, Apolice $apolicy): ApoliceResource
47 {
48     return new ApoliceResource($this->service->atualizar($apolicy,
49 ↵ $request->validated()));
50 }
51
52 public function destroy(Apolice $apolicy): Response
53 {
54     $this->service->excluir($apolicy);
55
56     return response()->noContent();
57 }
58
59 public function cotacao(CotacaoRequest $request): JsonResponse
60 {
61     return response()->json($this->service->cotar($request->validated()));
62 }

```

Código 12: app/Http/Controllers/Api/OpcoesController.php

```

1 <?php
2
3 namespace App\Http\Controllers\Api;
4
5 use App\Enums\Destino;
6 use App\Enums\Plano;
7 use App\Enums>StatusApolice;
8 use App\Http\Controllers\Controller;
9 use Illuminate\Http\JsonResponse;
10
11 class OpcoesController extends Controller
12 {
13     public function __invoke(): JsonResponse
14     {
15         return response()->json([
16             'planos' => array_map(fn (Plano $plano) => [
17                 'valor' => $plano->value,
18                 'label' => $plano->label(),
19                 'valorDiariaCentavos' => $plano->diariaCentavos(),
20                 'coberturaMedicaCentavos' => $plano->coberturaMedicaCentavos(),
21             ], Plano::cases()),
22             'destinos' => array_map(fn (Destino $destino) => [
23                 'valor' => $destino->value,
24                 'label' => $destino->label(),
25             ], Destino::cases()),
26             'status' => array_map(fn (StatusApolice $status) => [
27                 'valor' => $status->value,
28                 'label' => $status->label(),
29             ], StatusApolice::cases()),
30         ]);

```

```
31     }  
32 }
```

5.5 Service: regras de negócio

O `ApoliceService` concentra as regras. Pontos importantes:

- Recebe a **interface** `CalculadoraPremio` no construtor, não a classe concreta.
- `criar` e `atualizar` rodam em `DB::transaction`: segurado e apólice são gravados juntos ou nenhum é.
- `salvarSegurado` usa `updateOrCreate` pelo CPF, que reaproveita o segurado existente.
- `gerarNumero` consulta também as apólices excluídas (`withTrashed`) para nunca repetir um número.
- `listar` ordena pelas mais recentes e pagina 10 por página.

Código 13: `app/Services/ApoliceService.php`

```
1  <?php  
2  
3  namespace App\Services;  
4  
5  use App\Enums\Destino;  
6  use App\Enums\Plano;  
7  use App\Enums>StatusApolice;  
8  use App\Models\Apolice;  
9  use App\Models\Segurado;  
10 use App\Services\Premio\CalculadoraPremio;  
11 use Illuminate\Contracts\Pagination\LengthAwarePaginator;  
12 use Illuminate\Support\Carbon;  
13 use Illuminate\Support\Facades\DB;  
14 use Illuminate\Support\Str;  
15 use Illuminate\Validation\ValidationException;  
16  
17 class ApoliceService  
18 {  
19     public const POR_PAGINA = 10;  
20  
21     public function __construct(private readonly CalculadoraPremio $calculadora)  
22     {  
23     }  
24  
25     public function listar(?string $busca, ?string $status): LengthAwarePaginator  
26     {  
27         return Apolice::with('segurado')  
28             ->buscar($busca)  
29             ->when(StatusApolice::tryFrom((string) $status), fn ($query, $status) =>  
30                 ↳ $query->where('status', $status))  
31                 ->latest()  
32                 ->latest('id')  
33                 ->paginate(self::POR_PAGINA);  
34     }  
35  
36     public function resumo(): array  
37     {  
38         $ativas = Apolice::where('status', StatusApolice::Ativa);
```

```

39     return [
40         'total' => Apolice::count(),
41         'ativas' => $ativas->count(),
42         'premioAtivasCentavos' => (int) $ativas->sum('valor_premio_centavos'),
43     ];
44 }
45
46 public function criar(array $dados): Apolice
47 {
48     return DB::transaction(function () use ($dados) {
49         $apolice = Apolice::create([
50             ...$this->dadosDaApolice($dados),
51             'numero' => $this->gerarNumero(),
52             'segurado_id' => $this->salvarSegurado($dados)->id,
53             'status' => StatusApolice::Ativa,
54         ]);
55
56         return $apolice->load('segurado');
57     });
58 }
59
60 public function atualizar(Apolice $apolice, array $dados): Apolice
61 {
62     $novoStatus = StatusApolice::tryFrom($dados['status'] ?? '') ?? $apolice->status;
63
64     if ($apolice->status === StatusApolice::Cancelada && $novoStatus ===
65     ↪ StatusApolice::Cancelada) {
66         throw ValidationException::withMessages([
67             'status' => 'Apólice cancelada não pode ser alterada. Reative-a primeiro.',
68         ]);
69     }
70
71     return DB::transaction(function () use ($apolice, $dados, $novoStatus) {
72         $apolice->update([
73             ...$this->dadosDaApolice($dados),
74             'segurado_id' => $this->salvarSegurado($dados)->id,
75             'status' => $novoStatus,
76         ]);
77
78         return $apolice->load('segurado');
79     });
80 }
81
82 public function excluir(Apolice $apolice): void
83 {
84     $apolice->delete();
85 }
86
87 public function cotar(array $dados): array
88 {
89     $inicio = Carbon::parse($dados['inicioVigencia']);
90     $fim = Carbon::parse($dados['fimVigencia']);
91
92     return [
93         'valorPremioCentavos' => $this->premio($dados),
94         'dias' => (int) $inicio->diffInDays($fim) + 1,
95     ];
96 }
97
98 private function dadosDaApolice(array $dados): array
99 {
100     return [
101         'destino' => $dados['destino'],

```

```

101         'plano' => $dados['plano'],
102         'inicio_vigencia' => $dados['inicioVigencia'],
103         'fim_vigencia' => $dados['fimVigencia'],
104         'valor_premio_centavos' => $this->premio($dados),
105     ];
106 }
107
108 private function salvarSegurado(array $dados): Segurado
109 {
110     return Segurado::updateOrCreate(
111         ['cpf' => preg_replace('/\D/', '', $dados['seguradoCpf'])],
112         [
113             'nome' => trim($dados['seguradoNome']),
114             'email' => mb_strtolower(trim($dados['seguradoEmail'])),
115             'data_nascimento' => $dados['seguradoNascimento'],
116         ],
117     );
118 }
119
120 private function premio(array $dados): int
121 {
122     return $this->calculadora->calcular(
123         Plano::from($dados['plano']),
124         Destino::from($dados['destino']),
125         Carbon::parse($dados['inicioVigencia']),
126         Carbon::parse($dados['fimVigencia']),
127         Carbon::parse($dados['seguradoNascimento']),
128     );
129 }
130
131 private function gerarNumero(): string
132 {
133     do {
134         $numero = 'CRS-'.now()->year.'-'.Str::upper(bin2hex(random_bytes(4)));
135     } while (Apolice::withTrashed()->where('numero', $numero)->exists());
136
137     return $numero;
138 }
139 }

```

5.6 Calculadora do prêmio

A interface tem um único método. A implementação `CalculadoraPremioViagem` não acessa o banco nem o HTTP: recebe valores e devolve centavos, por isso é fácil de testar.

Código 14: `app/Services/Premio/CalculadoraPremio.php`

```

1 <?php
2
3 namespace App\Services\Premio;
4
5 use App\Enums\Destino;
6 use App\Enums\Plano;
7 use Carbon\CarbonInterface;
8
9 interface CalculadoraPremio
10 {
11     /** Retorna o valor do prêmio em centavos. */
12     public function calcular(
13         Plano $plano,
14         Destino $destino,

```

```

15     CarbonInterface $inicio,
16     CarbonInterface $fim,
17     CarbonInterface $dataNascimento,
18 ): int;
19 }

```

Código 15: app/Services/Premio/CalculadoraPremioViagem.php

```

1  <?php
2
3  namespace App\Services\Premio;
4
5  use App\Enums\Destino;
6  use App\Enums\Plano;
7  use Carbon\CarbonInterface;
8
9  class CalculadoraPremioViagem implements CalculadoraPremio
10 {
11     public function calcular(
12         Plano $plano,
13         Destino $destino,
14         CarbonInterface $inicio,
15         CarbonInterface $fim,
16         CarbonInterface $dataNascimento,
17     ): int {
18         $dias = (int) $inicio->diffInDays($fim) + 1;
19         $idade = (int) $dataNascimento->diffInYears($inicio);
20
21         $valor = $plano->diariaCentavos() * $dias;
22         $valor = $this->aplicarPercentual($valor, $destino->percentualRisco());
23
24         return $this->aplicarPercentual($valor, $this->percentualIdade($idade));
25     }
26
27     private function percentualIdade(int $idade): int
28     {
29         return match (true) {
30             $idade >= 75 => 250,
31             $idade >= 60 => 160,
32             default => 100,
33         };
34     }
35
36     private function aplicarPercentual(int $centavos, int $percentual): int
37     {
38         return intdiv($centavos * $percentual + 50, 100);
39     }
40 }

```

A ligação entre a interface e a implementação fica no service provider. Para trocar a regra de preço, basta alterar essa linha:

Código 16: app/Providers/AppServiceProvider.php

```

1  <?php
2
3  namespace App\Providers;
4
5  use App\Services\Premio\CalculadoraPremio;
6  use App\Services\Premio\CalculadoraPremioViagem;
7  use Illuminate\Http\Resources\Json\JsonResource;
8  use Illuminate\Support\ServiceProvider;

```

```
9
10 class AppServiceProvider extends ServiceProvider
11 {
12     /**
13      * Register any application services.
14      */
15     public function register(): void
16     {
17         $this->app->bind(CalculadoraPremio::class, CalculadoraPremioViagem::class);
18     }
19
20     /**
21      * Bootstrap any application services.
22      */
23     public function boot(): void
24     {
25         JsonResource::withoutWrapping();
26     }
27 }
```

5.7 Models

Os *casts* convertem as colunas automaticamente: *destino*, *plano* e *status* viram enums, e as datas viram objetos Carbon. O *scope* *buscar* procura o termo no número da apólice e no nome, e-mail e CPF do segurado; para o CPF, considera só os dígitos, então a busca funciona com ou sem máscara.

Código 17: app/Models/Segurado.php

```
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Database\Eloquent\Model;
7 use Illuminate\Database\Eloquent\Relations\HasMany;
8
9 class Segurado extends Model
10 {
11     use HasFactory;
12
13     protected $fillable = ['nome', 'cpf', 'email', 'data_nascimento'];
14
15     protected function casts(): array
16     {
17         return [
18             'data_nascimento' => 'date',
19         ];
20     }
21
22     public function apolices(): HasMany
23     {
24         return $this->hasMany(Apolice::class);
25     }
26
27     public function cpfFormatado(): string
28     {
29         return vsprintf('%s.%s.%s-%s', str_split($this->cpf, 3));
30     }
31 }
```

Código 18: app/Models/Apolice.php

```
1 <?php
2
3 namespace App\Models;
4
5 use App\Enums\Destino;
6 use App\Enums\Plano;
7 use App\Enums\StatusApolice;
8 use Illuminate\Database\Eloquent\Builder;
9 use Illuminate\Database\Eloquent\Factories\HasFactory;
10 use Illuminate\Database\Eloquent\Model;
11 use Illuminate\Database\Eloquent\Relations\BelongsTo;
12 use Illuminate\Database\Eloquent\SoftDeletes;
13
14 class Apolice extends Model
15 {
16     use HasFactory, SoftDeletes;
17
18     protected $table = 'apolices';
19
20     protected $fillable = [
21         'numero',
22         'segurado_id',
23         'destino',
24         'plano',
25         'inicio_vigencia',
26         'fim_vigencia',
27         'valor_premio_centavos',
28         'status',
29     ];
30
31     protected function casts(): array
32     {
33         return [
34             'destino' => Destino::class,
35             'plano' => Plano::class,
36             'status' => StatusApolice::class,
37             'inicio_vigencia' => 'date',
38             'fim_vigencia' => 'date',
39             'valor_premio_centavos' => 'integer',
40         ];
41     }
42
43     public function asegurado(): BelongsTo
44     {
45         return $this->belongsTo(Segurado::class);
46     }
47
48     public function dias(): int
49     {
50         return (int) $this->inicio_vigencia->diffInDays($this->fim_vigencia) + 1;
51     }
52
53     public function scopeBuscar(Builder $query, ?string $termo): Builder
54     {
55         if (blank($termo)) {
56             return $query;
57         }
58
59         $digitos = preg_replace('/\D/', '', $termo);
60
61         return $query->where(function (Builder $query) use ($termo, $digitos) {
62             $query->where('numero', 'like', "%{$termo}%")
```

```

63         ->orWhereHas('segurado', function (Builder $segurado) use ($termo,
        ↪ $digitos) {
64             $segurado->where('nome', 'like', "%{$termo}%")
65             ->orWhere('email', 'like', "%{$termo}%");
66
67             if ($digitos !== '') {
68                 $segurado->orWhere('cpf', 'like', "%{$digitos}%");
69             }
70         });
71     });
72 }
73 }

```

5.8 Resource: formato da resposta

O banco usa nomes em `snake_case`; a API responde em `camelCase`, o padrão do JavaScript. O resource faz essa tradução e já envia os textos de exibição (`planoLabel`, `statusLabel`...).

Código 19: `app/Http/Resources/ApoliceResource.php`

```

1  <?php
2
3  namespace App\Http\Resources;
4
5  use Illuminate\Http\Request;
6  use Illuminate\Http\Resources\Json\JsonResource;
7
8  /** @mixin \App\Models\Apolice */
9  class ApoliceResource extends JsonResource
10 {
11     public function toArray(Request $request): array
12     {
13         return [
14             'id' => $this->id,
15             'numero' => $this->numero,
16             'seguradoId' => $this->segurado->id,
17             'seguradoNome' => $this->segurado->nome,
18             'seguradoCpf' => $this->segurado->cpfFormatado(),
19             'seguradoEmail' => $this->segurado->email,
20             'seguradoNascimento' => $this->segurado->data_nascimento->toDateString(),
21             'destino' => $this->destino->value,
22             'destinoLabel' => $this->destino->label(),
23             'plano' => $this->plano->value,
24             'planoLabel' => $this->plano->label(),
25             'inicioVigencia' => $this->inicio_vigencia->toDateString(),
26             'fimVigencia' => $this->fim_vigencia->toDateString(),
27             'dias' => $this->dias(),
28             'valorPremioCentavos' => $this->valor_premio_centavos,
29             'status' => $this->status->value,
30             'statusLabel' => $this->status->label(),
31             'criadoEm' => $this->created_at->toIso8601String(),
32             'atualizadoEm' => $this->updated_at->eq($this->created_at) ? null :
        ↪ $this->updated_at->toIso8601String(),
33         ];
34     }
35 }

```

5.9 Tratamento de erros

Em bootstrap/app.php, toda rota /api/* responde erros em JSON. O 404 recebe uma mensagem em português, diferenciando apólice inexistente de rota inexistente.

Código 20: bootstrap/app.php

```

1 <?php
2
3 use Illuminate\Foundation\Application;
4 use Illuminate\Foundation\Configuration\Exceptions;
5 use Illuminate\Foundation\Configuration\Middleware;
6 use Illuminate\Http\Request;
7 use Symfony\Component\HttpKernel\Exception\NotFoundHttpException;
8
9 return Application::configure(basePath: dirname(__DIR__))
10     ->withRouting(
11         web: __DIR__.'/../routes/web.php',
12         api: __DIR__.'/../routes/api.php',
13         commands: __DIR__.'/../routes/console.php',
14         health: '/up',
15     )
16     ->withMiddleware(function (Middleware $middleware): void {
17         //
18     })
19     ->withExceptions(function (Exceptions $exceptions): void {
20         $exceptions->shouldRenderJsonWhen(fn (Request $request) => $request->is('api/*'));
21
22         $exceptions->render(function (NotFoundHttpException $e, Request $request) {
23             if ($request->is('api/*')) {
24                 $apolicyNaoEncontrada = str_contains($e->getMessage(), 'Apólice');
25
26                 return response()->json([
27                     'message' => $apolicyNaoEncontrada ? 'Apólice não encontrada.' : 'Rota
↵ não encontrada.',
28                     ], 404);
29             }
30         });
31     }->create();

```

6 Referência da API

Método	Rota	Descrição
GET	/api/apolices	Lista paginada. Parâmetros: busca, status, page
GET	/api/apolices/resumo	Totais para os cards da tela inicial
GET	/api/apolices/{id}	Detalhe de uma apólice
POST	/api/apolices	Cadastra (201)
PUT	/api/apolices/{id}	Edita e recalcula o prêmio
DELETE	/api/apolices/{id}	Exclusão lógica (204)
POST	/api/apolices/cotacao	Calcula o prêmio sem salvar
GET	/api/opcoes	Planos, destinos e status
GET	/api/health	Verificação de saúde

Status HTTP	Quando
200 OK	consulta, edição e cotação
201 Created	apólice cadastrada
204 No Content	apólice excluída
404 Not Found	apólice inexistente ou excluída; rota inexistente
422 Unprocessable Content	erro de validação, com a mensagem de cada campo

Código 21: Corpo de POST /api/apolices

```
{
  "seguradoNome": "Carlos Pereira",
  "seguradoCpf": "529.982.247-25",
  "seguradoEmail": "carlos@email.com",
  "seguradoNascimento": "1995-05-10",
  "destino": "europa",
  "plano": "plus",
  "inicioVigencia": "2026-10-01",
  "fimVigencia": "2026-10-10"
}
```

Código 22: Resposta 201 Created

```
{
  "id": 1,
  "numero": "CRS-2026-3B5CE1F5",
  "seguradoId": 1,
  "seguradoNome": "Carlos Pereira",
  "seguradoCpf": "529.982.247-25",
  "seguradoEmail": "carlos@email.com",
  "seguradoNascimento": "1995-05-10",
  "destino": "europa",
  "destinoLabel": "Europa",
  "plano": "plus",
  "planoLabel": "Plus",
  "inicioVigencia": "2026-10-01",
  "fimVigencia": "2026-10-10",
  "dias": 10,
  "valorPremioCentavos": 32370,
  "status": "ativa",
  "statusLabel": "Ativa",
  "criadoEm": "2026-09-23T13:00:00+00:00",
  "atualizadoEm": null
}
```

Código 23: Resposta 422 (erro de validação)

```
{
  "message": "CPF inválido. (e mais 1 erro)",
  "errors": {
    "seguradoCpf": ["CPF inválido."],
    "plano": ["Plano inválido."]
  }
}
```

A listagem segue o formato de paginação do Laravel: os registros ficam em `data`, e `meta` traz `current_page`, `last_page`, `per_page` e `total`.

7 Frontend

7.1 Telas e rotas

Rota	Tela
/apolices	ApoliceList: cards de resumo, busca, filtro, tabela e paginação
/apolices/nova	ApoliceCreate: formulário de cadastro
/apolices/:id	ApoliceDetails: detalhe e exclusão
/apolices/:id/editar	ApoliceEdit: formulário de edição

Código 24: frontend/src/App.jsx

```

1 import { Navigate, Route, Routes } from 'react-router-dom';
2 import Layout from './components/Layout';
3 import ApoliceCreate from './pages/ApoliceCreate';
4 import ApoliceDetails from './pages/ApoliceDetails';
5 import ApoliceEdit from './pages/ApoliceEdit';
6 import ApoliceList from './pages/ApoliceList';
7
8 export default function App() {
9   return (
10     <Routes>
11       <Route element={<Layout />}>
12         <Route index element={<Navigate to="/apolices" replace />} />
13         <Route path="/apolices" element={<ApoliceList />} />
14         <Route path="/apolices/nova" element={<ApoliceCreate />} />
15         <Route path="/apolices/:id" element={<ApoliceDetails />} />
16         <Route path="/apolices/:id/editar" element={<ApoliceEdit />} />
17         <Route path="*" element={<Navigate to="/apolices" replace />} />
18       </Route>
19     </Routes>
20   );
21 }

```

7.2 Comunicação com a API

Toda chamada passa pela função `request`. Ela define os cabeçalhos, converte o corpo para JSON, trata falha de conexão e, em caso de erro, lança um `ApiError` com a mensagem de cada campo. O Laravel envia os erros como lista (`["CPF inválido."]`), e o formulário mostra a primeira mensagem. A URL da API vem de `VITE_API_URL`, o que permite apontar para o App Service na Azure.

Código 25: frontend/src/api/client.js

```

1 const BASE_URL = (import.meta.env.VITE_API_URL ?? '/api').replace(/\/$/, '');
2
3 export class ApiError extends Error {
4   constructor(message, status, errors = {}) {
5     super(message);
6     this.status = status;
7     this.errors = errors;
8   }
9 }
10
11 // O Laravel devolve cada erro como lista (["CPF inválido."]); o formulário usa só a
   ↪ primeira mensagem.

```

```

12 function primeiraMensagemPorCampo(errors = {}) {
13   return Object.fromEntries(
14     Object.entries(errors).map(([campo, mensagens]) => [campo, Array.isArray(mensagens) ?
15       ↪ mensagens[0] : mensagens]),
16   );
17 }
18 export async function request(path, { method = 'GET', body, signal } = {}) {
19   let response;
20
21   try {
22     response = await fetch(`${BASE_URL}${path}`, {
23       method,
24       signal,
25       headers: {
26         Accept: 'application/json',
27         ...(body ? { 'Content-Type': 'application/json' } : {}),
28       },
29       body: body ? JSON.stringify(body) : undefined,
30     });
31   } catch (error) {
32     if (error.name === 'AbortError') throw error;
33     throw new ApiError('Não foi possível conectar à API.', 0);
34   }
35
36   if (response.status === 204) return null;
37
38   const data = await response.json().catch(() => null);
39
40   if (!response.ok) {
41     throw new ApiError(data?.message ?? 'Erro inesperado.', response.status,
42       ↪ primeiraMensagemPorCampo(data?.errors));
43   }
44
45   return data;
46 }

```

Código 26: frontend/src/api/apolices.js

```

1 import { request } from './client';
2
3 export const apolicesApi = {
4   listar: ({ busca, status, pagina = 1 } = {}, signal) => {
5     const params = new URLSearchParams({ page: String(pagina) });
6     if (busca) params.set('busca', busca);
7     if (status) params.set('status', status);
8
9     return request(`/apolices?${params}`, { signal });
10  },
11  resumo: () => request('/apolices/resumo'),
12  buscar: (id) => request(`/apolices/${id}`, {
13    criar: (dados) => request('/apolices', { method: 'POST', body: dados }),
14    atualizar: (id, dados) => request(`/apolices/${id}`, { method: 'PUT', body: dados }),
15    excluir: (id) => request(`/apolices/${id}`, { method: 'DELETE' }),
16    cotar: (dados, signal) => request('/apolices/cotacao', { method: 'POST', body: dados,
17      ↪ signal }),
18    opcoes: () => request('/opcoes'),
19  });

```

7.3 Cotação em tempo real

No formulário, o hook `useCotacao` chama `POST /api/apolices/cotacao` quando todos os campos estão preenchidos. Ele espera 400 ms sem digitação (*debounce*) e cancela a requisição anterior com `AbortController`, para que uma resposta antiga não sobrescreva a mais nova. A lista usa a mesma técnica na busca, com 300 ms.

Código 27: frontend/src/components/ApoliceForm.jsx (trecho)

```
175 function useCotacao(form) {
176   const [cotacao, setCotacao] = useState(null);
177   const completo = CAMPOS_COTACAO.every((campo) => form[campo]);
178   const chave = CAMPOS_COTACAO.map((campo) => form[campo]).join('|');
179
180   useEffect(() => {
181     if (!completo) {
182       setCotacao(null);
183       return undefined;
184     }
185
186     const controller = new AbortController();
187     const timer = setTimeout(() => {
188       apolicesApi
189         .cotar(form, controller.signal)
190         .then(setCotacao)
191         .catch((error) => {
192           if (error.name !== 'AbortError') setCotacao(null);
193         });
194     }, 400);
195
196     return () => {
197       clearTimeout(timer);
198       controller.abort();
199     };
200   }, [chave, completo]);
201
202   return cotacao;
203 }
```

7.4 Opções e formatação

`useOpcoes` busca planos, destinos e status uma única vez e guarda em cache na memória. `format.js` converte centavos para reais (`Intl.NumberFormat`), formata datas e aplica a máscara do CPF durante a digitação.

Código 28: frontend/src/hooks/useOpcoes.js

```
1 import { useEffect, useState } from 'react';
2 import { apolicesApi } from '../api/apolices';
3
4 let cache = null;
5
6 export function useOpcoes() {
7   const [opcoes, setOpcoes] = useState(cache);
8   const [erro, setErro] = useState(null);
9
10  useEffect(() => {
11    if (cache) return;
12
13    apolicesApi
```

```

14     .opcoes()
15     .then((dados) => {
16         cache = dados;
17         setOpcoes(dados);
18     })
19     .catch(setErro);
20 }, []);
21
22 return { opcoes, erro };
23 }

```

Código 29: frontend/src/utils/format.js

```

1  const moeda = new Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL' });
2
3  export const formatarMoeda = (centavos) => moeda.format((centavos ?? 0) / 100);
4
5  export function formatarData(isoDate) {
6      if (!isoDate) return '--';
7      const [ano, mes, dia] = isoDate.slice(0, 10).split('-');
8      return `${dia}/${mes}/${ano}`;
9  }
10
11 export function formatarDataHora(iso) {
12     if (!iso) return '--';
13     return new Date(iso).toLocaleString('pt-BR', { dateStyle: 'short', timeStyle: 'short' });
14 }
15
16 export function mascaraCpf(valor) {
17     return valor
18         .replace(/\D/g, '')
19         .slice(0, 11)
20         .replace(/(\d{3})(\d)/, '$1.$2')
21         .replace(/(\d{3})(\d)/, '$1.$2')
22         .replace(/(\d{3})(\d{1,2})$/, '$1-$2');
23 }

```

8 SOLID e Clean Code

Princípio	Aplicação no projeto
S – Responsabilidade única	O Form Request valida, o Controller trata o HTTP, o Service aplica a regra de negócio, a Calculadora só calcula e o Resource só formata a resposta.
O – Aberto/fechado	Novo plano ou destino é um novo case no enum. Uma nova regra de preço (promoção, por exemplo) é uma nova classe que implementa CalculadoraPremio , sem alterar o Service.
L – Substituição de Liskov	Qualquer implementação de CalculadoraPremio pode substituir a atual, pois o Service depende apenas do contrato (recebe dados, devolve centavos).
I – Segregação de interfaces	A interface da calculadora tem um único método, calcular .
D – Inversão de dependência	O Service depende da abstração; o AppServiceProvider decide qual implementação o container injeta.

Clean Code:

- Nomes em português, iguais aos termos do negócio (*apólice, segurado, prêmio, vigência*).
- Métodos curtos e com um objetivo: `salvarSegurado`, `gerarNumero`, `dadosDaApolice`.
- Sem números mágicos: valores de negócio ficam nos enums e em constantes (`VIGENCIA_MAXIMA_DIAS`, `POR_PAGINA`).
- Regras que dependem de várias colunas (como o status cancelado) ficam no Service, e não espalhadas pelo front.

9 Testes automatizados

São 25 testes com 61 verificações, todos passando.

9.1 Testes de unidade

- `CalculadoraPremioViagemTest`: 5 cenários combinando plano, destino e faixa etária, com o valor esperado em centavos. Usa `DataProvider`, então cada cenário aparece como um teste separado.
- `CpfTest`: CPFs válidos com e sem máscara; CPFs vazios, repetidos, com dígito errado ou incompletos.

Código 30: `tests/Unit/CalculadoraPremioViagemTest.php`

```
1 <?php
2
3 namespace Tests\Unit;
4
5 use App\Enums\Destino;
6 use App\Enums\Plano;
7 use App\Services\Premio\CalculadoraPremioViagem;
8 use Illuminate\Support\Carbon;
9 use PHPUnit\Framework\Attributes\DataProvider;
10 use PHPUnit\Framework\TestCase;
11
12 class CalculadoraPremioViagemTest extends TestCase
13 {
14     #[DataProvider('cenarios')]
15     public function test_calcula_premio_por_plano_destino_dias_e_idade(Plano $plano,
16         ↪ Destino $destino, string $nascimento, int $esperado): void
17     {
18         $premio = (new CalculadoraPremioViagem())->calcular(
19             $plano,
20             $destino,
21             Carbon::parse('2026-10-01'),
22             Carbon::parse('2026-10-10'),
23             Carbon::parse($nascimento),
24         );
25
26         $this->assertSame($esperado, $premio);
27     }
28
29     public static function cenarios(): array
30     {
31         return [
32             'plus, europa, adulto' => [Plano::Plus, Destino::Europa, '1990-01-01', 32370],
33             'premium, europa, adulto' => [Plano::Premium, Destino::Europa, '1990-01-01',
34                 ↪ 51870],
```

```

33         'essencial, américa do sul, adulto' => [Plano::Essencial,
    ↪ Destino::AmericaDoSul, '1990-01-01', 12900],
34         'plus, europa, 66 anos' => [Plano::Plus, Destino::Europa, '1960-01-01', 51792],
35         'essencial, américa do sul, 81 anos' => [Plano::Essencial,
    ↪ Destino::AmericaDoSul, '1945-01-01', 32250],
36     ];
37 }
38 }

```

9.2 Testes da API

ApoliceApiTest faz requisições HTTP reais contra a aplicação, com o banco recriado a cada teste (RefreshDatabase) e a data fixada em 20/09/2026 (Carbon::setTestNow), para que os resultados não dependam do dia em que o teste roda.

Teste	O que garante
cria_apolice_calculando_o_premio	201, prêmio correto, número no formato, CPF salvo sem máscara
reaproveita_o_segurado_pelo_cpf	duas apólices do mesmo CPF usam um único segurado
lista_com_paginacao_busca_e_filtro	paginação, busca por CPF e nome, filtro por status
edita_e_recalcula_o_premio	troca de plano atualiza o prêmio
apolice_cancelada_so_pode_ser_reativada	422 ao alterar cancelada; reativação permitida
exclusao_logica	204, depois 404, e o registro continua no banco
valida_os_campos_em_portugues	mensagens de CPF e plano inválidos
vigencia_nao_pode_comecar_no_passado...	início no passado, mais de 365 dias, fim antes do início
apolice_ja_em_vigor_pode_ser_editada...	edição sem mudar o início, mesmo após a data de hoje
cotacao_nao_persiste	a cotação calcula e não grava nada
resumo_ignora_canceladas_e_excluidas	totais corretos nos cards
opcoes_e_health	rotas auxiliares
rota_inexistente	404 com mensagem em português

Código 31: Rodando os testes

```

cd backend
php artisan test

```

10 Execução

10.1 Docker (modo principal)

Código 32: Subindo a aplicação

```

docker compose up -d --build          # MySQL, API e frontend
docker compose exec api php artisan db:seed # dados de exemplo (opcional)

```

A aplicação fica em `http://localhost:3000` e a API em `http://localhost:8000/api`. Ao subir, o container da API aguarda o MySQL e executa as migrations.

10.2 Sem Docker

Localmente a API usa SQLite, então não é preciso instalar MySQL.

Código 33: Execução local

```
cd backend
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve

cd frontend          # em outro terminal
npm install
npm run dev           # http://localhost:5173
```

10.3 Publicação na Azure

1. Criar o **Azure Database for MySQL – Flexible Server** e o banco `coris_seguros`.
2. Criar o **App Service** (Linux, PHP 8.3), configurar as variáveis de ambiente (`APP_KEY`, `DB_*`, `MYSQL_ATTR_SSL_CA`) e o comando de inicialização `bash /home/site/wwwroot/azure/startup.sh`, que aponta o nginx para a pasta `public` e roda as migrations.
3. Opcionalmente, guardar a senha do banco no **Key Vault** e referenciá-la nas variáveis do App Service.
4. Criar o **Static Web App** apontando para a pasta `frontend`, com `VITE_API_URL` igual à URL do App Service seguida de `/api`.

11 Próximos passos

- Autenticação de usuários com Laravel Sanctum e perfis de acesso (operador e gestor), já que os dados dos segurados são pessoais (LGPD).
- Endosso: histórico de cada alteração feita em uma apólice emitida.
- Infraestrutura como código (Bicep) para criar os recursos da Azure.
- Testes no frontend.
- Pipeline de CI rodando os testes antes do deploy.